

JNose: Java Test Smell Detector

Tássio Virgínio
tassio.virginio@ifto.edu.br
UFBA, Bahia, Brazil
IFTO, Tocantins, Brazil

Luana Martins
martins.luana@ufba.br
UFBA, Bahia, Brazil

Larissa Rocha
lrsoares@uefs.br
UFBA, Bahia, Brazil
UEFS, Bahia, Brazil

Railana Santana
railana.santana@ufba.br
UFBA, Bahia, Brazil

Adriana Cruz
adriana.cruz@estudante.ufla.br
UFLA, MG, Brazil

Heitor Costa
heitor@ufla.br
UFLA, MG, Brazil

Ivan Machado
ivan.machado@ufba.br
UFBA, Bahia, Brazil

ABSTRACT

Several strategies have been proposed for test quality measurement and analysis. Code coverage is likely the most widely used one. It enables to verify the ability of a test case to cover as many source code branches as possible. Although code coverage has been widely used, novel strategies have been recently employed. It is the case of test smells analysis, which has been introduced as an affordable strategy to evaluate the quality of test code. Test smells are poor design choices in implementation, and their occurrence in test code might reduce the quality of test suites. Test smells identification is clearly dependent on tool support, otherwise it could become a cost-ineffective strategy. However, as far as we know, there is no tool that combines code coverage and test smells to address test quality measurement. In this work, we present the JNose Test, a tool aimed to analyze test suite quality in the perspective of test smells. JNose Test detects code coverage and software evolution metrics and a set of test smells throughout software versions.

CCS CONCEPTS

• **Software and its engineering** → **Software testing and debugging**; *Empirical software validation*.

KEYWORDS

Quality of Tests, Test Suite Evolution, Test Smells, Code Coverage

ACM Reference Format:

Tássio Virgínio, Luana Martins, Larissa Rocha, Railana Santana, Adriana Cruz, Heitor Costa, and Ivan Machado. 2020. JNose: Java Test Smell Detector. In *34th Brazilian Symposium on Software Engineering (SBES '20)*, October 21–23, 2020, Natal, Brazil. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3422392.3422499>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SBES '20, October 21–23, 2020, Natal, Brazil

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-8753-8/20/09...\$15.00

<https://doi.org/10.1145/3422392.3422499>

1 INTRODUCTION

A software testing consultancy recently reported on an analysis of 606 software failures from 314 companies, aimed to understand the impact of test failures on commercial and financial fields [11]. They found that software failures affected over 3.6 billion people, resulting in a financial loss of US\$1.7 trillion for companies in various sectors (finance, retail, consumer technology, services and transportation) in the year of 2017. This example illustrates the need for employing software testing techniques in software development processes, as they could anticipate the detection, correction, and the likely effects, of *bugs* still during implementation (or even when existing functionalities are under evolution) [4, 8, 10].

In a well-defined software engineering process, test code is expected to co-evolve together with production code, as high-quality test code is essential to ease the maintenance and evolution of both, production and test code [5, 14]. However, it might be both time-consuming and cost-ineffective [5, 14].

Several approaches have been proposed in the literature to assess the quality of test suites. For example, code coverage measurement has been widely used to check the quality of automated tests. It measures the test suite quality based on how much it covers structural elements, such as functions, instructions, branches, and lines of code [3]. Nonetheless, even with a high code coverage, the test code might encompass poor design choices in their implementation, the so-called test smells. The presence of smells in test code may reduce the quality of test suites and, consequently, the quality of the production code [2]. Additionally, poorly-written tests can be difficult to comprehend, which make it onerous for testers to maintain the code and detect faults [1, 4].

The software testing literature has introduced a set of tools focused on validating the quality of test suites, mainly through metrics analysis. For example, *CodeCover*¹ is an open source Java tool for code coverage executed via both graphical user interface (with Eclipse IDE) and command-line; *tsDetect*² is a command-line tool for test smells detection. Other tools use code coverage results to predict test smells, such as *TeReDetect* [7] and *TeCReVis* [6]. Generally, these tools have many different data outputs, which might be hard for testers to establish a relationship between code coverage and internal test code quality. Moreover, there are several types of test smells that have not been investigated in conjunction with code coverage yet, but could also provide opportunities to improve test code quality.

¹ Available at: <http://codecover.org/>

² Available at: <https://testsmells.github.io/>

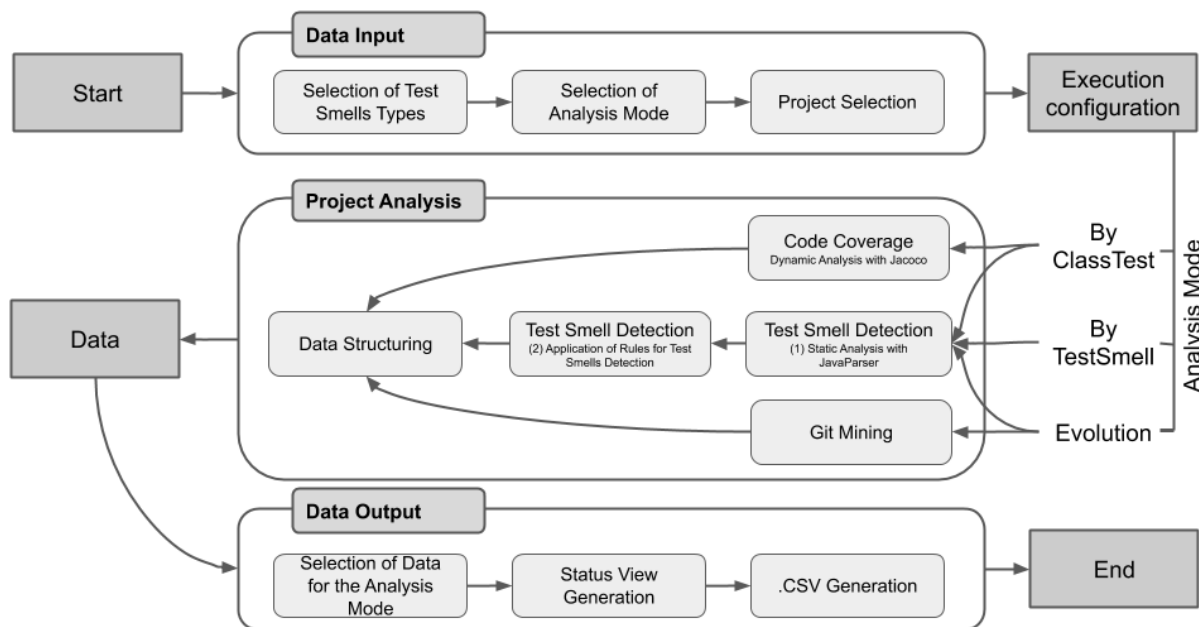


Figure 1: Schematic overview of the JNose Test tool and its main features

In this paper, we introduce the JNose Test, a tool to analyze the quality of test suites, in the perspective of test smells. JNose Test provides an automated test strategy focused on (i) identifying possible test design flaws, (ii) analyzing the quality evolution of a software project, and (iii) reducing the effort for performing quality assurance of a test suite.

JNose Test integrates a conceptual framework under development by an inter-institutional researchers' team, which consists of test smells prevention, identification, refactoring, and visualization to improve the test code quality. RAIDE³ and TSVizzEvolution⁴ tools are part of this framework.

The remainder of this paper is structured as follows. Section 2 presents an overview of the JNose Test and its architecture. Section 3 illustrates an example of use. Section 4 describes a comparison between our proposed tool and similar ones. Section 5 draws concluding remarks.

2 JNOSE TEST TOOL

The JNose Test tool⁵ enables test code quality analysis through test smells detection and code coverage calculation, over several versions of a software project. Therefore, it is possible to compare whether a project's test quality has either improved or declined throughout its life cycle. The JNose Test operation involves three key processes, as Figure 1 shows: (i) **Data Input**, receives the settings for the tool execution, i.e., the list of types of test smells, analysis mode (code coverage, test smells detection, and evolution), and the project to be analyzed; (ii) **Project Analysis**, performs the analysis of the project according to the analysis mode selected; and

(iii) **Data Output**, presents the execution status and generates a .csv file with the results of the analysis.

2.1 Processes Description

The JNose Test pursue two installation requirements: Java Development Kit (JDK) 1.8 and Maven 3 (or superior). Upon installation, the user would be able to use Jetty (embedded on Maven), besides building and running the JNose Test. Figure 2 presents the main screen of the JNose Test.

Once started, the tool requires the user to configure the **Data Input** (Figure 1). First, the user selects the analysis mode, i.e., *by ClassTest*, *by TestSmells* or *Evolution* (Figure 2 - Step 1). Each mode provides a field where the user should paste the project path to be analyzed. By default, the tool detects twenty-one types of test smells, but it can also be configured by the user (Figure 2 - Step 2).

After finishing the execution configuration, the tool starts the **Project Analysis** (Figure 1). It considers the analysis mode selected by user, described as follows:

(1) By ClassTest. In the **Data Input** process, the user can enable the coverage metrics calculation and specify the corresponding paths of the local projects (Figure 3 - Steps 1 and 2). Then, to analyze the project by test class, the **Project Analysis** process executes the *Test Smell Detection* module, and optionally executes the *Code Coverage* module. These two modules do not require information about other project versions, as the analysis is performed on a local project. To track the execution status, the **Data Output** process provides a view to make the user aware of which project analysis has already been done (Figure 3 - Step 3). Finally, the tool generates a .csv file with the results of the data analysis by test class (Figure 3 - Step 4). Each .csv row corresponds to a given test class, and

³Available at <https://github.com/raideplugin/Plugin-RAIDE>

⁴Available at <https://github.com/AdrianaPriscilaSantos/TSVizzEvolution>

⁵Available at: <https://github.com/tassiovirginio/jnose.git>

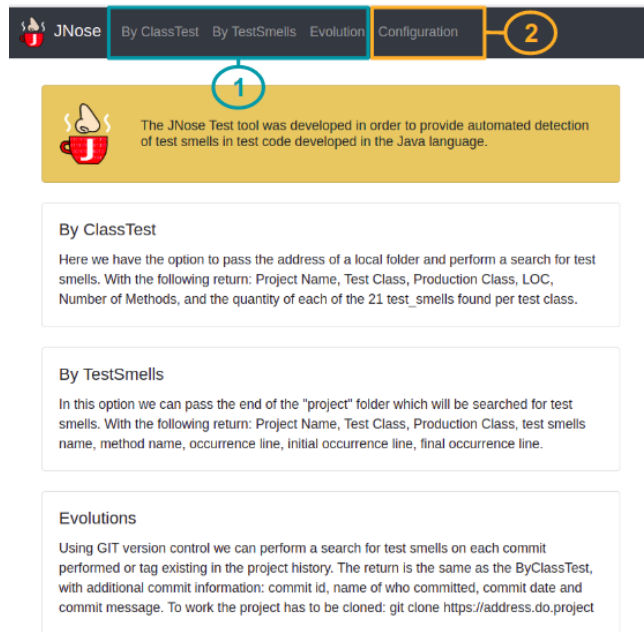


Figure 2: Main view of the JNose Test

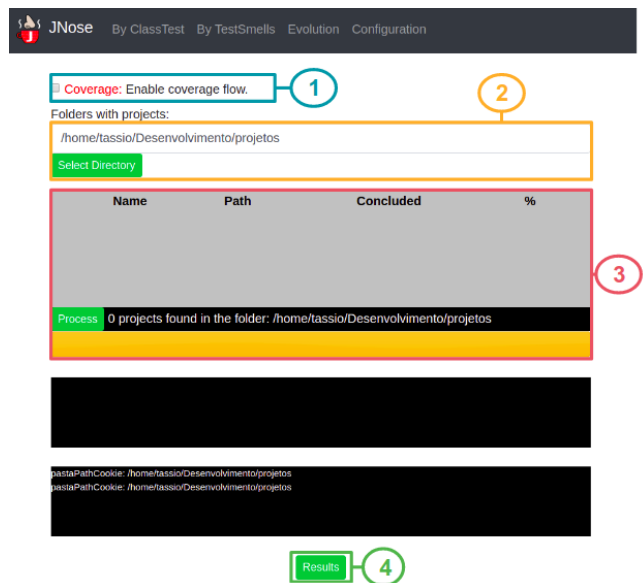


Figure 3: View of the execution by TestClass

each column represents the type of parameter collected: project name, test class and production class location, twenty-one types of test smells, the number of test class lines, the number of test methods, and five columns with coverage data (Table 1).

(2) **By TestSmell.** To analyze the project by test smell, the **Project Analysis** process would only execute the *Test Smell Detection* module. Similarly to the *ClassTest* analysis, during the **Data Input** process, the user needs to specify the projects' local paths (Figure 4 - Step 1). To track the execution status, the **Data Output** also

Table 1: Data returned by TestClass

#	Parameter	Description
1	App	Project name
2	TestFileName	Test class name
3	ProductionFileName	Production class name
4	LOC	Lines of code in a test class
5	Number of methods	Number of test methods
6-26	Test Smells	Number of test smells by type
27-31	Coverage Metrics	Percentage of a coverage metrics

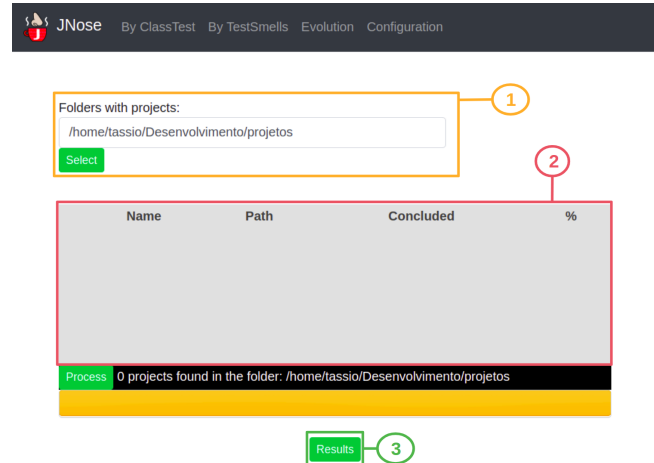


Figure 4: View of the execution by TestSmell

Table 2: Data returned by TestSmell

#	Parameter	Description
1	App	Project name
2	TestFileName	Test class name
3	ProductionFileName	production class name
4	Test Smell	Test smell name
5	MethodLocation	Name of the method with a test smell
6	LineLocation	Method location of a test smell in number of line

provides the progress view (Figure 4 - Step 2). Finally, it generates a .csv file with the results of the data analysis by test smells (Figure 4 - Step 3). Unlike the previous analysis, we provide the exact method where the test smell is located. Each row of the .csv represents a test smell, and it has six columns to show the type of parameter collected, the project name, the test class and the production class location, the test smell name, the method of the test smell location in number of line, and the method name (Table 2).

(3) **Evolution.** To analyze the project by versions, the **Project Analysis** process executes the *Git Mining* module. As this module requires information about project versions, the analysis is performed based on a project clone. Therefore, during the **Data Input**, the user needs to specify the path of the cloned git project, as well as the search he wants to apply, either by commits or by tags (Figure 5 - Steps 1 and 2). To track the execution status, the

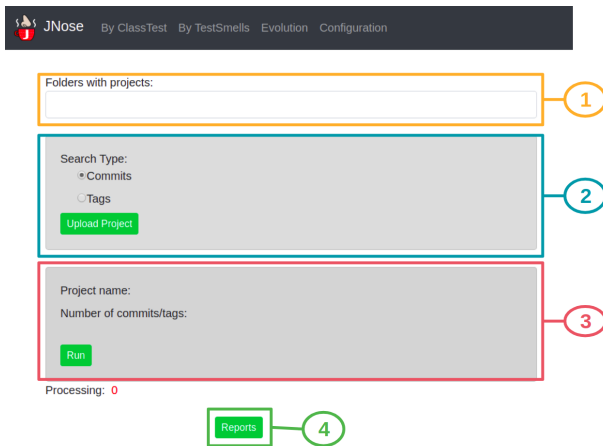


Figure 5: View of the execution by Evolution

Data Output process provides the same view as presented in the previous analysis (Figure 5 - Step 3). At the end, the process creates a .csv file containing the results of the data analysis by test smells (Figure 5 - Step 4).

This analysis provides the test smell detection by test class for each project version, in addition to data about the author who committed the test smell. The .csv rows represent the test classes by commit; and the columns have the following collected parameters: project name, test class and production class location, number of test smells, commit identification, authorship, date and message (Table 3). To find out who committed the test smell, the tool automatically calculates the authorship of a test smell by guilt, i.e., for the tester who last modified the method (and did not fixed it), assuming that he was aware of the problem and attribute to him the authorship.

2.2 Tool Architecture

The JNose Test tool is licensed under the GNU General Public License. The tool is implemented as a Java project and comprises four packages: (i) **core**, responsible for the detection of test smells and coverage metrics; (ii) **page**, responsible for presenting the web pages and their content; (iii) **dto**, encompassing the classes used in data transfer (Data Transfer Object); and (iv) **util**, responsible for identifying the tests and production classes, and saving results into the .csv files.

The **core** package implements the **Project Analysis**, which was split into three other packages, as follows:

- **Coverage.** It applies the business rules necessary to calculate code coverage. Basically, it identifies the test classes related to a production class and runs the JaCoCo library⁶. JaCoCo is an open-source library to calculate code coverage in Java language. JaCoCo performs a dynamic analysis of the production code branches (BC), instructions (IC), lines (LC), complexity (CC), and methods (MC) to determine which one is either missed or covered by the test code [12];
- **TestSmellDetector.** It performs a static analysis of the test code through an AST (Abstract Syntax Tree) generated by

⁶Available at: <https://www.eclemma.org/jacoco/>

Table 3: Data returned by Evolution

#	Parameter	Description
1	App	Project name
2	TestFileName	Test class name
3	ProductionFileName	production class name
4	CommitID	Commit identification
5	User Commit	User name who did the commit
6	Commit Date	Date when the commit was made
7	Commit MSG	The text message committed
8	Test Smells	Number of type smells by type

JavaParser⁷. Then, it extracts information about the code structure to apply the rules for the test smells detection, and it collects additional information about the location and number of test smells. The detection rules were reused from the tsDetect tool, which has twenty-one rules for test smells detection that cover the following smells [9, 12]: Assertion Roulette (AR), Conditional Test Logic (CTL), Constructor Initialization (CI), Default Test (DT), Dependent Test (DpT), Duplicate Assert (DA), Eager Test (ET), Empty Test (EmT), Exception Catching Throwing (ECT), General Fixture (GF), Ignored Test (IgT), Lazy Test (LT), Magic Number Test (MNT), Mystery Guest, Redundant Print, Redundant Assertion (RA), Resource Optimism (RO), Sensitive Equality (SE), Sleepy Test (ST), verbose Test (VT), and Unknown Test (UT).

The JNose Test interface was implemented in the **page** package based on the Apache Wicket⁸, a framework for the development of web applications in Java. We also used HTML5 and CSS3 to develop the web pages. This package implements the **Data Input**, as Figure 1 shows.

The **util** package implements two utility classes responsible for finding the test and production classes and export the results to the .csv files. For each analysis mode, a different type of report is generated. This package implements the **Data Output** (Figure 1).

In the **dto** package, we have the classes used for the transfer of data among the layers of the projects. This package implements **Data Input, Project Analysis, Data Output** (Figure 1).

In this project, the Apache Maven⁹ was used to manage the dependencies, build and run the JNose Test tool. Furthermore, the JNose Test execution uses parallel processes, i.e., for each uploaded project, the tool creates a new thread, for each test class the tool creates another thread, and so on. With the parallel processing, the JNose Test tool can be used to analyze a massive set of projects in a short time period [12].

3 EXAMPLE OF USE

In a previous work, we carried out an experimental study to verify the correlation between the coverage metrics and test smells. To perform this study, we selected eleven software projects, in which we collected twenty-one test smells and five coverage metrics using the JNose Test [12, 13].

⁷Available at: <https://javaparser.org/>

⁸Available at: <https://wicket.apache.org/>

⁹Available at: <https://maven.apache.org/>

In this section, we present an example considering the different types of analysis mode supported by JNose Test. We use the commons-io project¹⁰ (release 2.6-RC3), which is a library of utilities to assist I/O development. Each supported mode is discussed next.

3.1 TestClass analysis

We ran the JNose Test by TestClass to analyze which type of test smells achieved the highest diffusion over the commons-io project. Thus, the following steps were taken: (i) select all types of test smells; (ii) select the project path; and (iii) enable code coverage. The tool returned 58 test classes. To understand the test smell type diffusion, we checked the number of classes where each test smell was present. For example, the *Exception Catching Throwing* (ECT) test smell was present in 23 classes, followed by *Assertion Roulette* (AR) in 17 test classes, and *Eager Test* (ET) in 16 test classes. Each type of test smell can occur many times in a test class. These three types of test smells presented the highest occurrence in the project, counting 316, 175, and 157 times, respectively.

Table 4 shows five test classes which have the highest number of ECT, AR and ET types of test smells. For example, the test class ProxyCollectionWriterTest contains the highest number of those three test smells. Additionally, most test classes achieved good code coverage, when considering the IC, LC and MC coverage metrics (>70%). Therefore, even with a high coverage, the test code might present a low-quality.

3.2 TestSmell Analysis

Once we found that ECT, AR, and ET have the highest diffusion numbers in the commons-io project test classes, we may want to improve the test code quality by fixing the problems. Then, we executed the JNose Test by TestSmell by taking the following steps: (i) select the ECT, AR, and ET test smells; and (ii) select the project path. Table 5 shows the ProxyCollectionWriterTest test class, which has 23 test methods, but filtered to just show ECT, ET, and AR.

Listing 1 shows an excerpt of the ProxyCollectionWriterTest test class with the testArrayIOExceptionOnAppendChar1() test method (lines 39-53). We observe that the assertEquals() method is called twice within the test method (lines 50-51). Each one checks a different condition, but there is no explanation message for them. Thus, if the test method fails, there is no clue to identify which assertion caused the failure. This issue refers to the AR test smell. Moreover, those assertions are also related to ECT, because they may fail when a specific exception occurs. Furthermore, a test method is supposed to check just one method of the production class; otherwise, the code has an ET test smell, (ProxyCollectionWriter() on line 43 and append() on line 46).

3.3 Evolution Analysis

The evolution analysis might help us to identify whether the test code quality of the commons-io has improved over time. To perform this analysis, we should take the following steps: (i) select all test smells; (ii) select the analysis by commit; and (ii) select the project path. From the beginning until the release 2.6-RC3, the project has

¹⁰ Available at: <https://github.com/apache/commons-io>

```

37 public class ProxyCollectionWriterTest{
38
39     @Test
40     public void testArrayIOExceptionOnAppendChar1() throws
41         IOException {
42         final Writer badW = new BrokenWriter();
43         final StringWriter goodW = mock(StringWriter.class);
44         final ProxyCollectionWriter tw = new
45             ProxyCollectionWriter(badW, goodW, null);
46         final char data = 'A';
47         try {
48             tw.append(data);
49             fail("Expected "+IOException.class.getName());
50         } catch (final IOException e) {
51             verify(goodW).append(data);
52             assertEquals(1, e.getCauseList().size());
53             assertEquals(0, e.getCause(0, IOException.class).getIndex());
54         }
55     }
56 }

```

Listing 1: ProxyCollectionWriterTest test class

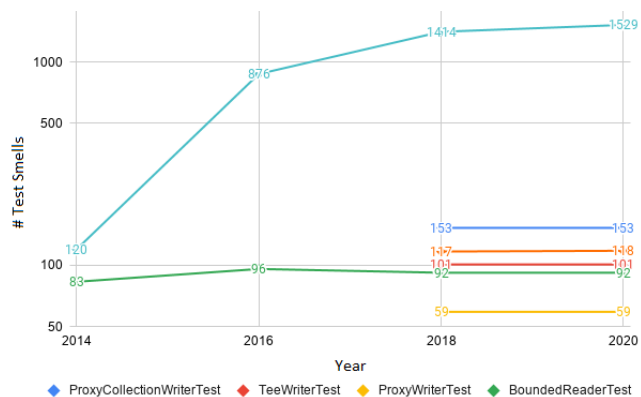


Figure 6: Evolution of the commons-io project and classes with high diffusion of test smells

2,337 commits, 52 releases, and 56 contributors. We filtered the results by the five test classes with more occurrences of ECT, ET and AR, as Table 6 shows.

Figure 6 shows the evolution of these classes and the project. The test classes ProxyCollectionWriterTest, TeeWriterTest, and ProxyWriterTest are stable, as no test smell was either inserted or fixed. However, the BoundedReaderTest class presented novel test smells during 2014-2016, and fixed during 2016-2020. We could observe that the number of test smells increases over the time, which might indicate that people involved in the development of the project test suite have not worked to get rid of test smells. In addition, authorship is calculated by fault, so the authors from this example might not have inserted all these detected test smells.

4 COMPARISON WITH SIMILAR TOOLS

In the past few years, many tools have been proposed to detect test smells. In particular, two of them correlate code coverage with test smells detection, TeReDetect [7] and TeCreVis [6]. Those tools support test suites developed in JUnit, and they detect test code duplication through code coverage analysis, held by CodeCover. Our tool uses a rule-based detection of test smells instead of a code coverage-based detection. In particular, our tool presents the code

Table 4: Classes with high diffusion of test smell - by TestClass

App	TesFileName	ProductionFileName	LOC	Met	UT	IgT	RO	MNT	RA	SE	VT	ST	LT	DA	ET	AR	CTL	CI	DT	EmT	ECT	GF	MG	PS	DpT	IC	BC	LC	CC	MC
commons-io	ProxyCollectionTest	ProxyCollection	448	23	1	0	0	21	0	1	0	0	61	1	23	21	1	0	0	0	23	0	0	0	0	72	0	76	100	100
commons-io	TeeWriteTest	TeeWrite	448	23	1	0	0	21	0	1	0	0	30	1	2	21	1	0	0	0	23	0	0	0	0	100	0	100	100	100
commons-io	ProxyWriteTest	ProxyWrite	275	21	3	0	0	0	0	8	0	0	23	0	4	0	0	0	0	0	21	0	0	0	0	83	0	87	93	93
commons-io	BoundedReaderTest	BoundedReader	246	22	1	1	1	10	0	0	0	0	48	1	8	3	2	0	0	0	16	0	1	0	0	100	100	100	100	100
commons-io	EndianUtilsTest	EndianUtils	316	22	1	0	0	13	0	0	0	0	46	8	20	15	1	0	0	0	14	0	0	0	0	100	100	100	100	100

Table 5: Test Smells location in ProxyCollectionWriterTest

App	TesFileName	ProductionFile	TestSmell	MethodLocationName	Line
commons-io	ProxyCollectionTest	ProxyCollection	AR	testArrayIOExceptionOnAppendChar1	40
commons-io	ProxyCollectionTest	ProxyCollection	AR	testArrayIOExceptionOnAppendChar2	56
commons-io	ProxyCollectionTest	ProxyCollection	AR	testArrayIOExceptionOnAppendCharSe	72
commons-io	ProxyCollectionTest	ProxyCollection	ET	testArrayIOExceptionOnAppendChar1	40
commons-io	ProxyCollectionTest	ProxyCollection	ET	testArrayIOExceptionOnAppendChar2	56
commons-io	ProxyCollectionTest	ProxyCollection	ET	testArrayIOExceptionOnAppendCharSe	72
commons-io	ProxyCollectionTest	ProxyCollection	ECT	testArrayIOExceptionOnAppendChar1	40
commons-io	ProxyCollectionTest	ProxyCollection	ECT	testArrayIOExceptionOnAppendChar2	56
commons-io	ProxyCollectionTest	ProxyCollection	ECT	testArrayIOExceptionOnAppendCharSe	72

Table 6: Classes with high diffusion of test smell - Evolution

App	TesFileName	ProductionFile	TestSmell	CommitID	CommitName	CommitDate
commons-io	ProxyCollectionWrite	ProxyCollection	153	b739ce7c	Adam Retter	03:39:47 2020
commons-io	ProxyCollectionWrite	ProxyCollection	153	bc36041	David Georg	00:09:03 2018
commons-io	TeeWriteTest	TeeWrite	101	b739ce7c	Adam Retter	03:39:47 2020
commons-io	TeeWriteTest	TeeWrite	101	bc36041	David Georg	00:09:03 2018
commons-io	ProxyWriteTest	ProxyWrite	59	b739ce7c	Adam Retter	03:39:47 2020
commons-io	ProxyWriteTest	ProxyWrite	59	bc36041	David Georg	00:09:03 2018
commons-io	BoundedReaderTest	BoundedReader	92	b739ce7c	Adam Retter	03:39:47 2020
commons-io	BoundedReaderTest	BoundedReader	96	51f13c84	Kristian Rose	15:36:15 2016
commons-io	BoundedReaderTest	BoundedReader	83	9a9b8385	Gary D. Greg	01:17:05 2014
commons-io	EndianUtilsTest	EndianUtils	118	b739ce7c	Adam Retter	03:39:47 2020
commons-io	EndianUtilsTest	EndianUtils	117	8940848G	Gary D. Greg	18:47:06 2018

The "Commit MSG" column was hidden because it is not used in the example.

coverage and types of test smells as complementary metrics for the JUnit test suites quality evaluation.

TsDetect, the test smells detection state-of-the-art tool [1] identifies twenty-one test smell types. It returns a boolean value to indicate whether a particular test smell appears in the test code. We reused the tsDetect detection rules, because it presents a precision score ranging from 85% to 100% and a recall score from 90% to 100% [9]. However, we extended it in several respects. For example, our tool provides the number of each type of test smells identified in a test class, and the method line and name where each test smell is located. Moreover, it supports the test suite analysis through several project versions, by mining the Git for providing information about when and by who the test smells have been committed.

5 CONCLUSION

This paper presents the JNose Test, a tool for test code quality analysis. It detects test smells and calculates code coverage for Java projects. JNose detects twenty-one types of test smells, which can optionally be correlated with the code coverage to assess the test suite quality.

We investigate test smells on (i) test class (by TestClass), which returns the amount of each type of test smell detected in a test class; (ii) over several project versions (by Evolution), which returns the information about who and when the test smell was inserted in the test code; (iii) and the detection can be applied to identify the exact location of a test smell (by TestSmell), which returns the method location of a smell.

In future work, new features could be implemented. For example, a feature to analyze the amount of test smells added by each user

of a Git repository in a version of the project, or even a feature to show the test code that meets the test smell. It is also possible to implement new measures to support software test quality analysis.

ACKNOWLEDGMENTS

This research was partially funded by FAPESB grant JCB0060/2016 and INES 2.0, CNPq grant 465614/ 2014-0.

REFERENCES

- [1] Gabriele Bavota, Abdallah Qusef, Rocco Oliveto, Andrea De Lucia, and Dave Binkley. 2015. Are test smells really harmful? An empirical study. *Empirical Software Engineering* 20, 4 (2015), 1052–1094.
- [2] Arie Deursen, Leon M.F. Moonen, A. Bergh, and Gerard Kok. 2001. Refactoring Test Code. In *Refactoring Test Code*. CWI (Centre for Mathematics and Computer Science), Amsterdam, The Netherlands, The Netherlands.
- [3] Rahul Gopinath, Carlos Jensen, and Alex Groce. 2014. Code Coverage for Suite Evaluation by Developers. In *Proceedings of the 36th International Conference on Software Engineering (ICSE 2014)*. ACM, New York, NY, USA, 72–82.
- [4] Giovanni Grano, Fabio Palomba, Dario Di Nucci, Andrea De Lucia, and Harald C. Gall. 2019. Scented since the beginning: On the diffuseness of test smells in automatically generated test code. *Journal of Systems and Software* 156 (2019), 312–327.
- [5] Dayne Guerra Calle, Julien Delplanque, and Stéphane Ducasse. 2019. Exposing Test Analysis Results with DrTests. In *International Workshop on Smalltalk Technologies*. HAL, Cologne, Germany, 1–5. <https://hal.inria.fr/hal-02404040>
- [6] Negar Koochakzadeh and Vahid Garousi. 2010. TeCreVis: A Tool for Test Coverage and Test Redundancy Visualization. In *Testing – Practice and Research Techniques*, Leonardo Bottaci and Gordon Fraser (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 129–136.
- [7] Koochakzadeh Negar and Vahid Garousi. 2010. A Tester-Assisted Methodology for Test Redundancy Detection. *Advances in Software Engineering* 2010 (01 2010). <https://doi.org/10.1155/2010/932686>
- [8] Fabio Palomba, Andy Zaidman, and Andrea De Lucia. 2018. Automatic Test Smell Detection Using Information Retrieval Techniques. In *IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, Madrid, Spain, 311–322.
- [9] Anthony Peruma, Khalid Almalki, Christian D. Newman, Mohamed Wiem Mkaouer, Ali Ouni, and Fabio Palomba. 2019. On the Distribution of Test Smells in Open Source Android Applications: An Exploratory Study. In *Proceedings of the 29th Annual International Conference on Computer Science and Software Engineering (CASCON '19)*. IBM Corp., Riverton, NJ, USA.
- [10] Davide Spadini, Fabio Palomba, Andy Zaidman, Magiel Bruntink, and Alberto Bacchelli. 2018. On the Relation of Test Smells to Software Code Quality. In *International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, Madrid, Spain, 1–12.
- [11] Tricentis. 2018. Software Fail Watch: 5th Edition. <https://www.tricentis.com/resources/software-fail-watch-5th-edition/> Last access: June 2020.
- [12] Tássio Virginio, Santana Railana, Luana Almeida Martins, Larissa Rocha Soares, Heitor Costa, and Ivan Machado. 2019. On the Influence of Test Smells on Test Coverage. In *Proceedings of the XXXIII Brazilian Symposium on Software Engineering (SBES 2019)*. ACM, New York, NY, USA, 467–471.
- [13] Tássio Virginio, Luana Martins, Larissa Rocha Soares, Santana Railana, Heitor Costa, and Ivan Machado. 2020. An empirical study of automatically-generated tests from the perspective of test smells. In *Proceedings of the XXXIV Brazilian Symposium on Software Engineering (SBES 2020)*. ACM, New York, NY, USA, 5.
- [14] Vahid Garousi Yusifoglu, Yasaman Amannejad, and Aysu Betin Can. 2015. Software test-code engineering: A systematic mapping. *Information and Software Technology* 58 (2015), 123 – 147.